

3. Regularization and Optimization

Regularization and Optimization

흐름: 지난 강의 복습 → Regularization → Optimization → SGD의 문제 → Momentum·RMSProp·Adam·AdamW → Learning Rate Schedule → Second-Order Optimization

- Regularization: overfitting 방지. generalization 위주.
- Optimzation: 최적 W 찾기.

1. Regularization

1.1 Data Loss와 Regularization

Regularization

λ = regularization strength (hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss: Model predictions should match training data}} + \underbrace{\lambda R(W)}_{\text{Regularization: Prevent the model from doing too well on training data}}$$

→ performance + data loss 영향 X

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

→ penalty 항 추가 → W 자체 제약

More complex:

Dropout

Batch normalization

Stochastic depth, fractional pooling, etc

같은 값이면 0 근처는 아닐 때 / 큰 값보다 작은 값 여러 개로 분산
값이 0 근처일 때 / sparse weight

- 전체 loss $L(W)$
- **Data loss:** 모델의 예측이 training data와 일치하도록 함.
- **Regularization:** 모델이 training data에 지나치게 맞춰지는 것을 억제함.

- $R(W)$: regularization 항
 - λ : regularization strength 결정하는 hyperparameter
-

1.2 Prefer Simpler Models

→ **Occam's Razor**: 여러 가설이 데이터를 설명한다면 더 단순한 가설을 선호

Regularization의 목적:

- weight에 대한 선호를 표현
 - 모델을 단순하게 만들어 test data에서 잘 작동하도록 함
 - loss에 curvature를 추가해 optimization을 개선
-

1.3 Regularization 종류

- 간단한 예시: L2 Regularization, L1 Regularization, Elastic Net (L1 + L2)
 - weight에 더 큰 penalty 부여
 - 복잡한 예시: Dropout, Batch Normalization, Stochastic Depth, Fractional Pooling
-

1.4 Regularization → Weight 선호 표현

- L1: 값이 0 근처일 때 / sparse weight
- L2: 값은 작지만 0 근처는 아닐 때 / 큰 값보다 작은 값 여러 개로 분산

e.g)

$x=[1,1,1,1]$

$w_1=[1,0,0,0]$, $w_2=[0.25,0.25,0.25,0.25]$

- L2 penalty
 - $R(w_1)=1^2=1$
 - $R(w_2) = 4(0.25^2) = 0.25$
 - L1 penalty
 - $R(w_1)=1+0+0+0=1$
 - $R(w_2) = 4(0.25) = 1$
-

3. Optimization

3.1 Loss Landscape

Loss landscape: 모든 가능한 W 에 대한 loss를 지형으로 표현

Optimization: Loss Landscape에서 loss가 낮아지는 방향으로 W 를 이동시키는 과정

3.2 Strategy #1 — Random Search

→ 임의의 W 를 여러 개 생성한 뒤 가장 낮은 loss를 만드는 W 를 선택한다.

임의 탐색은 weight 공간의 방향 정보를 사용하지 않는다.

3.3 Strategy #2 — Follow the Slope

함수

- 1차원 함수: 함수의 derivative = 현재 위치에서 함수가 변하는 기울기
- 다차원 함수: gradient = 각 차원에 대한 partial derivative를 모은 벡터.

특징

- 임의의 방향에서의 기울기: 해당 방향과 gradient의 dot product
 - 가장 빠르게 증가하는 방향: gradient 방향
 - 가장 빠르게 감소하는 방향: **negative gradient 방향**
-

4. Gradient 계산

4.1 Numerical Gradient

한 차원의 weight를 작은 값 h 만큼 변화시켜 loss의 변화를 측정한다.

특징: 작성하기 쉬움. 근삿값. 모든 차원을 하나씩 변화시켜야 하므로 느림

4.2 Analytic Gradient

미분법을 이용해 loss function의 gradient를 직접 계산한다.

특징: 정확함. 빠름, 구현 과정에서 오류가 발생하기 쉬움.

실제 사용

gradient check: 두 결과를 비교하는 과정

- 학습에는 analytic gradient 사용
 - 구현 검증에는 numerical gradient 사용
-

5. Gradient Descent

현재 weight에서 negative gradient 방향으로 이동한다.

- α : learning rate 또는 step size
- gradient의 반대 방향으로 이동하면 loss가 감소함.

한계: 데이터 수 N 이 크면, 매번 전체 합을 계산하는 비용이 크다.

6. Stochastic Gradient Descent

전체 데이터 대신 **minibatch**를 사용해 합을 근사한다.

일반적인 minibatch 크기: 32, 64, 128

Problem #1 — Poor Conditioning

Loss가 한 방향에서는 빠르게 변하고 다른 방향에서는 느리게 변하는 경우:

- steep direction에서는 좌우로 크게 진동함.
- shallow direction에서는 이동이 느림. → 최솟값을 향한 진행이 비효율적임.

Hessian의 가장 큰 singular value와 가장 작은 singular value의 비율이 크면, loss function의 **condition number**가 높다.(gradient descent가 최적점으로 가기 까다롭다)

Problem #2 — Local Minima와 Saddle Point

Gradient가 0이 되는 지점에서는 gradient descent의 이동이 멈춘다.

- Local minimum
- Saddle point

고차원 공간에서는 local minimum보다 saddle point가 더 흔하다.

Saddle point는 어떤 방향에서는 최솟값이고, 다른 방향에서는 최댓값인 지점이다.

Problem #3 — Gradient Noise

SGD는 minibatch로 전체 gradient를 근사한다.

- minibatch마다 포함되는 샘플이 다름.
 - 계산되는 gradient에 noise가 생김.
 - update 방향이 흔들릴 수 있음.
-

7. SGD + Momentum

이전 iteration의 이동 방향을 **velocity**로 누적한다.

효과

- 이전 iteration과 비슷한 방향으로 계속 이동
- shallow direction의 velocity가 누적됨.
- steep direction의 진동이 완화됨.
- noisy gradient의 영향을 평균화함.
- saddle point와 poor conditioning 상황에서 SGD보다 빠르게 진행할 수 있음.

Momentum은 여러 방식으로 표기되지만 같은 sequence를 만든다.

8. RMSProp

과거 gradient 제공의 이동평균을 이용해 각 parameter의 gradient 크기를 조정한다.

특징

- parameter마다 서로 다른 크기의 update 적용
- **Per-parameter learning rate, Adaptive learning rate**

Loss landscape에서의 변화

- steep direction: 큰 gradient 제공이 누적되어 이동이 억제됨.
 - flat direction: 상대적으로 이동이 가속됨.
-

9. Adam, AdamW

Adam

Momentum + RMSProp 함께 사용한다.

한계

L2 regularization 항이 gradient에 포함된다.

- regularization이 포함된 gradient를 사용해 moment를 계산함.
- parameter마다 adaptive scaling을 거치므로 L2 항도 함께 scaling됨.

AdamW

AdamW의 Weight Decay

weight decay를 moment 계산과 분리한다.

- Adam optimizer로 gradient update 계산
- moment 계산이 끝난 뒤 weight decay를 별도로 적용

→ Standard Adam의 L2 regularization과 AdamW의 weight decay는 adaptive optimizer에서 동일하게 작동하지 않는다.

10. Learning Rate Schedules

hyperparameter: learning rate

서로 다른 learning rate도 schedule과 학습 조건에 따라 좋은 결과를 낼 수 있다.

Learning Rate Decay

미리 정한 epoch에서 learning rate를 줄인다.

ResNet 예시:

- Epoch 30: learning rate에 0.1을 곱함.
- Epoch 60: 다시 0.1을 곱함.
- Epoch 90: 다시 0.1을 곱함.

종류

- Cosine Decay
- Linear Decay: Learning rate가 시간에 따라 선형적으로 감소한다.
- Inverse Square Root Decay: 초기에 빠르게 감소하고 이후에는 완만하게 감소한다.

11. First-Order Optimization

현재 위치에서 gradient를 이용해 loss의 선형 근사를 만든다.

1. Gradient로 linear approximation 계산
2. 해당 근사를 감소시키는 방향으로 이동

Gradient Descent, SGD, Momentum, RMSProp, Adam은 first-order 정보를 사용한다.

12. Second-Order Optimization

Gradient와 Hessian을 이용해 loss를 quadratic function으로 근사한다.

Second-Order Taylor Expansion

Critical point를 계산하면 Newton update를 얻는다.

Deep Learning에서 사용하기 어려운 이유: 계산량과 메모리 비용이 매우 크다.

13. Optimizer 선택

권고

- **Adam 또는 AdamW**
 - 여러 경우에 기본 선택으로 사용 가능
 - constant learning rate에서도 비교적 잘 작동함.
- **SGD + Momentum**
 - Adam보다 좋은 성능을 낼 수 있음.
 - learning rate와 schedule을 더 많이 조정해야 할 수 있음.
- **Full-batch update가 가능한 경우**
 - first-order보다 높은 차수의 optimization 방법도 고려